Université Mohammed V de Rabat
École Nationale Supérieure d'Informatique et d'Analyse des Systèmes
Département Ingénierie des Systèmes Embarqués
Filière Smart Systems Engineering

Tutorial 3 : Créer un service BLE sur l'Arduino Portenta H7 avec MicroPython (aioble) dans OpenMV IDE

Séries de tuto sur Portenta H7, lecture 3 « Faire parler la station »

Pr.Hassan BERBIA/Pr.Aissam BERRAHOU

2 novembre 2026

Objectifs du Tutorial

Dans ce tutoriel, vous allez donner la parole à l'unité de décision de la station EcoTri 4.0. La Portenta H7 va publier sa présence en Bluetooth Low Energy, exposer un service GATT « Décision » et dialoguer avec le smartphone de l'opérateur : recevoir des commandes, piloter ses LED et pousser des messages de décision par notification. Chaque étape est reliée aux notions du cours (GAP, GATT, CCCD, MTU, débit).
À la fin de ce tutoriel, vous serez capable de :

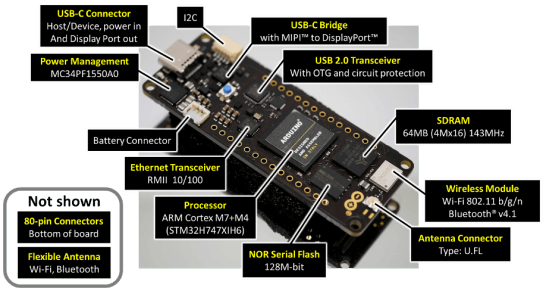
- Vérifier la pile Bluetooth de la Portenta et balayer les appareils BLE voisins (rôle central).
- Publier un service GATT avec des caractéristiques read, write et notify (rôle périphérique).
- Tester un service avec l'application **nRF Connect** : connexion, découverte, écriture, abonnement.
- Piloter les LED de la carte par commandes BLE et renvoyer un accusé de réception.
- Notifier un message de décision JSON issu de la caméra, en parallèle de la capture (`asyncio`).
- Négocier le MTU et mesurer le débit de notifications.

Durée indicative : 1 h 15. Ce tutoriel prépare le TP3 (image fragmentée XIAO → Portenta et jalon J3).

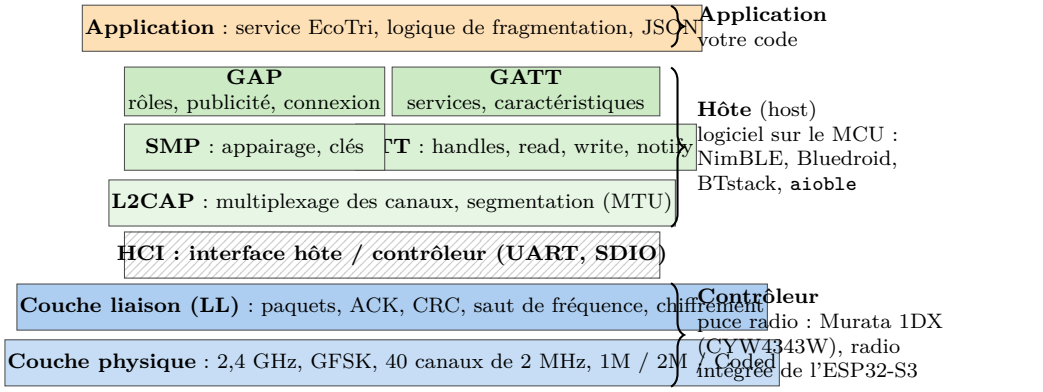
1 Introduction : le Bluetooth Low Energy sur la Portenta H7

La **Portenta H7** intègre le module sans fil **Murata 1DX**, construit autour de la puce Cypress **CYW4343W** : Wi-Fi 802.11 b/g/n et **Bluetooth 5.1**. Dans la pile BLE, ce module joue le rôle de **contrôleur** (radio, couche liaison) ; l'**hôte** (L2CAP, ATT, GATT, GAP) s'exécute dans le firmware OpenMV, sur le Cortex-M7. Vous le pilotez en MicroPython grâce à deux modules :

- `bluetooth` : l'interface bas niveau de MicroPython, fondée sur des événements (fonction `irq`). Puissante mais verbeuse.
- `aioble` : une surcouche écrite en Python au-dessus de `bluetooth`, qui transforme chaque opération en coroutine `asyncio` (`await aioble.advertise()`, `await caract.written()`…). C'est elle que nous utilisons.



La Portenta H7 : le module Murata 1DX (Wi-Fi + Bluetooth) et le connecteur U.FL de l'antenne sont situés à côté du STM32H747.



d'après Bluetooth Core Specification v5.4, vol. 1, partie A ; Townsend et al., Getting Started with BLE, O'Reilly

Rappel du cours : la pile BLE. Votre script MicroPython se situe tout en haut ; `aioble` et le firmware OpenMV forment l'hôte ; le module Murata est le contrôleur.

Ce que nous allons construire

Le service **Décision** de la Portenta, tel que spécifié dans le cours (premier jet du jalon J3) :

Élément	UUID	Propriétés	Contenu
Service Décision	`e0c00010-7b1a-4b2c-9d3e-ec0712a00000`	–	serveur : Portenta
DECISION	`e0c00011-7b1a-4b2c-9d3e-ec0712a00000`	read, notify	message JSON
COMMANDE	`e0c00012-7b1a-4b2c-9d3e-ec0712a00000`	write	`LED:R`, `PING`…

2 Pré-requis

Matériel

- Carte **Arduino Portenta H7** avec son **Vision Shield** (étape 5) et son antenne (connecteur U.FL, fournie avec la carte).
- Câble USB-C (données).
- Un **smartphone** Android ou iOS avec Bluetooth activé.

Logiciel

1. **OpenMV IDE (version 4.0 ou plus)** et le firmware OpenMV à jour sur la Portenta (Tutorial 1).
2. **nRF Connect for Mobile** (Nordic Semiconductor), gratuit sur Google Play et l'App Store.
3. Documentation : module `aioble` sur docs.openmv.io et exemples **File** → **Examples** → **Bluetooth** dans OpenMV IDE.

Astuce

L'antenne doit être branchée : sans elle, la portée tombe à quelques centimètres et les déconnexions se multiplient. Sous Android, nRF Connect demande les autorisations « Appareils à proximité » et parfois « Localisation » : acceptez-les, sinon le balayage reste vide.

3 Étapes détaillées du tutoriel

Étape 1 : Vérifier la pile Bluetooth

```
import bluetooth, sys

ble = bluetooth.BLE()
ble.active(True)                          # démarre le contrôleur (module Murata)
mac = ble.config("mac")[1]                # (type d'adresse, adresse)
print("Firmware :", sys.version)
print("Adresse BLE :", ":".join("%02X" % b for b in mac))
print("MTU local max :", ble.config("mtu"))

try:
    import aioble
    print("aioble disponible")
except ImportError:
    print("aioble absent : copiez le dossier aioble de micropython-lib sur la carte")
```

— Notez l'**adresse BLE** : vous la retrouverez dans nRF Connect.

— Si `aioble` est absent, copiez le dossier `aioble/` du dépôt micropython-lib (répertoire `micropython/bluetooth/aioble`) à la racine du disque USB de la carte.

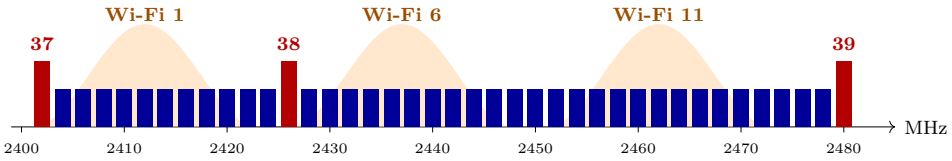
Étape 2 : Balayer les appareils voisins (rôle central)

Avant de publier, écoutons : la Portenta joue ici le rôle **observer/central** et liste les publicités reçues pendant 5 s.

```python
import asyncio, aioble

async def balayer():
    vus = {}
    # fenêtre = intervalle : balayage continu ; active=True demande la SCAN_RSP
    async with aioble.scan(5000, interval_us=30000, window_us=30000, active=True) as
    scanner:
        async for r in scanner:
            cle = bytes(r.device.addr)
            if cle not in vus or r.name():
                vus[cle] = (r.name() or "?", r.rssi, [str(u) for u in r.services()])
    for addr, (nom, rssi, svcs) in sorted(vus.items(), key=lambda x: -x[1][1]):
        print("%-22s %4d dBm  %s" % (nom, rssi, svcs))
    print(len(vus), "appareils")

asyncio.run(balayer())
```



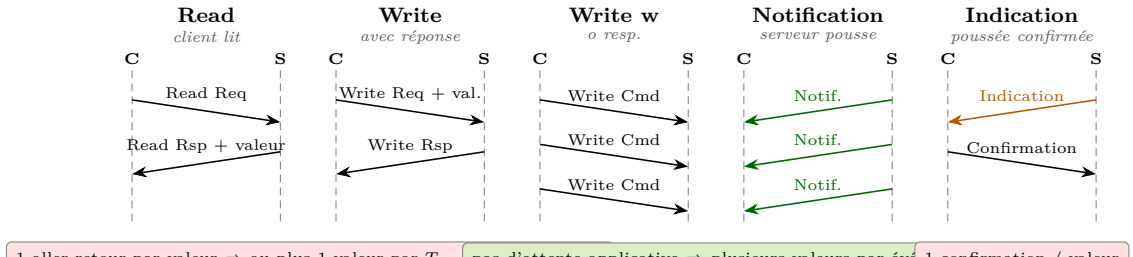
Les canaux de publicité sont placés entre les canaux Wi-Fi 1, 6 et 11 les plus utilisés ; en connexion, le saut de fréquence évite les canaux brouillés.

Le balayage écoute successivement les trois canaux de publicité 37, 38 et 39. La fenêtre de 30 ms toutes les 30 ms correspond à une écoute continue.

Question

Combien d'appareils voyez-vous dans la salle ? Approchez puis éloignez votre téléphone (en publicité, par exemple avec l'onglet *Advertiser* de nRF Connect) : comment varie le RSSI ? Pourquoi un même appareil apparaît-il parfois sans nom (indice : nom transmis dans la SCAN_RSP, ou adresse aléatoire qui change) ?

Étape 3 : Publier le service Décision (rôle périphérique)



Dans tous les cas la couche liaison acquitte chaque paquet (ACK + CRC) : ce qui manque à « notify », c'est l'accusé de réception ***de l'application****.*

Les opérations ATT. DECISION utilise read et notify ; COMMANDE utilise write.

```python
import asyncio, aioble, bluetooth

BASE = "-7b1a-4b2c-9d3e-ec0712a00000"
```

```
SVC_DEC = bluetooth.UUID("e0c00010" + BASE)
CAR_DEC = bluetooth.UUID("e0c00011" + BASE)
CAR_CMD = bluetooth.UUID("e0c00012" + BASE)

service = aioble.Service(SVC_DEC)
decision = aioble.Characteristic(service, CAR_DEC, read=True, notify=True)
commande = aioble.Characteristic(service, CAR_CMD, write=True, capture=True)
aioble.register_services(service)
decision.write(b'{"v":1,"etat":"pret"}')        # valeur initiale (lecture)

async def periph():
    while True:
        print("Publicité en cours...")
        conn = await aioble.advertise(
            100_000,                          # intervalle de publicité : 100 ms
            name="EcoTri-Portenta",
            services=[SVC_DEC])
        print("Connecté à", conn.device)
        await conn.disconnected(timeout_ms=None)   # attend la déconnexion
        print("Déconnecté")

asyncio.run(periph())
```

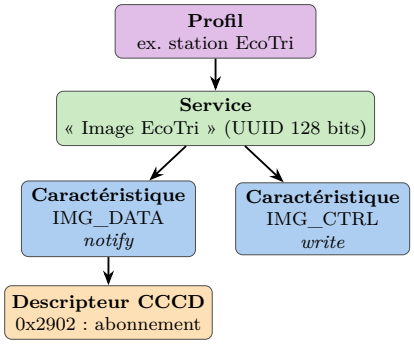
Dans nRF Connect :

1. Onglet **Scanner** : repérez `EcoTri-Portenta`, puis **Connect**.
2. Dépliez le service *Unknown Service* `E0C00010-...` : vous voyez deux caractéristiques et leurs propriétés (*READ, NOTIFY* et *WRITE*).
3. Appuyez sur la flèche de lecture de DECISION : la valeur `{"v":1,"etat":"pret"}` s'affiche (choisissez l'affichage *UTF-8*).

Capture à insérer :
source/nrf_scan.png

Capture à insérer :
source/nrf_service.png

nRF Connect : la Portenta dans la liste des appareils (gauche) et le service Décision découvert après connexion (droite). Insérez vos propres captures d'écran.



Handle	Type (UUID)	Valeur	Perm.
0x0010	0x2800 Service primaire	UUID du service	R
0x0011	0x2803 Décl. caract.	notify, h=0x0012, UUID	R
0x0012	UUID IMG_DATA	fragment (≤ MTU−3 o)	–
0x0013	0x2902 CCCD	0x0001 = notif. ON	R/W
0x0014	0x2803 Décl. caract.	write, h=0x0015, UUID	R
0x0015	UUID IMG_CTRL	commande / NACK	W

Tout est « attribut » : un handle de 16 bits, un type (UUID), une valeur et des permissions. Le client ne manipule que des handles ; les UUID ne servent qu'à la découverte.

La table d'attributs que nRF Connect reconstruit par découverte. Comparez les handles affichés par votre téléphone avec ce schéma (ici pour le service Image du TP3).

Question

Relevez dans nRF Connect le **handle** de chaque caractéristique et la présence d'un descripteur `0x2902` (CCCD) sous DECISION. Pourquoi COMMANDE n'en a-t-elle pas ?

Étape 4 : Recevoir des commandes et piloter les LED

La caractéristique COMMANDE a été créée avec `capture=True` : chaque écriture est mise en file et `await commande.written()` rend le couple (connexion, données). On ajoute une tâche qui interprète les commandes texte.

```
from pyb import LED
import json

LEDS = {"R": LED(1), "G": LED(2), "B": LED(3)}

def leds_off():
    for l in LEDS.values(): l.off()

async def commandes():
    while True:
        conn, data = await commande.written()
        cmd = bytes(data).decode().strip().upper()
        print("Commande reçue :", cmd)
        if cmd.startswith("LED:"):
            leds_off()
            for c in cmd[4:]:
                if c in LEDS: LEDS[c].on()
            rep = {"ack": cmd}
        elif cmd == "PING":
            rep = {"ack": "PONG"}
        else:
            rep = {"err": "commande inconnue", "cmd": cmd}
        # met à jour la valeur ET notifie les clients abonnés
        decision.write(json.dumps(rep).encode(), send_update=True)

async def main():
    await asyncio.gather(periph(), commandes())

asyncio.run(main())
```

1. Dans nRF Connect, abonnez-vous à DECISION (icône aux trois flèches : le client écrit `0x0001` dans le CCCD).
2. Écrivez sur COMMANDE (flèche montante, type *TEXT*) : `LED:R`, `LED:GB`, `PING`, `FOO`.
3. Observez les LED et la notification de réponse dans nRF Connect.

Astuce

Désabonnez-vous de DECISION puis renvoyez `PING` : la LED réagit mais aucune notification n'arrive. C'est le rôle du CCCD vu en cours : sans abonnement, le serveur n'a pas le droit de notifier.

Étape 5 : Notifier une décision issue de la caméra

La Portenta doit capturer et notifier **en même temps**. La capture est bloquante : on la place dans une tâche `asyncio` qui rend la main au gestionnaire BLE à chaque tour (`await`

`asyncio.sleep_ms(0)`). Pour ce tutoriel, la « classe » est simulée par la luminosité moyenne de la zone centrale ; elle sera remplacée par le classifieur à règles (lecture 2) puis par le CNN (lecture 5).

```
import sensor, time

sensor.reset()
sensor.set_pixformat(sensor.GRAYSCALE)
sensor.set_framesize(sensor.QVGA)
sensor.skip_frames(time=1500)
ROI = (110, 70, 100, 100)

async def vision():
    n = 0
    clock = time.clock()
    while True:
        clock.tick()
        img = sensor.snapshot()
        L = img.get_statistics(roi=ROI).mean()
        n += 1
        if n % 10 == 0:                          # une décision toutes les 10 images
            cls = "metal" if L > 150 else ("carton" if L > 90 else "vide")
            msg = {"v": 1, "id": n // 10, "t": time.ticks_ms(), "cls": cls,
                   "L": L, "fps": round(clock.fps(), 1)}
            data = json.dumps(msg).encode()
            decision.write(data, send_update=True)
            print(len(data), "o :", data)
        await asyncio.sleep_ms(0)                # laisse la main à la pile BLE

async def main():
    await asyncio.gather(periph(), commandes(), vision())

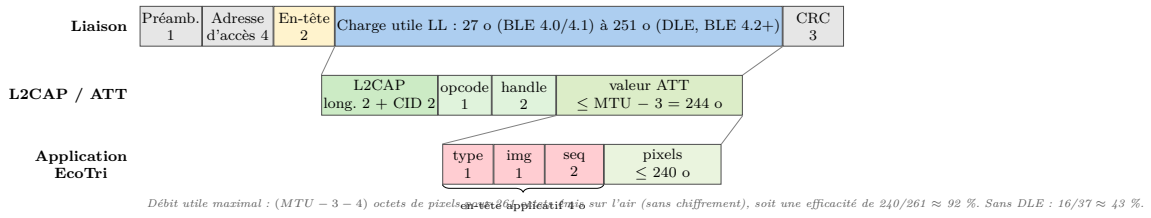
asyncio.run(main())
```

Question

Quelle est la taille des messages JSON affichée par le script ? Avec le MTU par défaut (23 octets), nRF Connect reçoit-il le message complet ? Que se passe-t-il pour les octets au-delà de 20 ? Gardez la réponse en tête pour l'étape 6.

Étape 6 : Négocier le MTU

Une notification ne transporte que $MTU - 3$ octets : les 20 octets par défaut tronquent le JSON. Deux solutions : agrandir le MTU, ou raccourcir le message (format binaire, vu en cours). On agrandit le MTU :



Rappel : la valeur ATT d'une notification ne peut dépasser $MTU - 3$ octets (1 octet d'opcode et 2 de handle).

```
aioble.config(mtu=247)                   # MTU maximal annoncé par la Portenta
```

```
async def periph():
    while True:
        conn = await aioble.advertise(100_000, name="EcoTri-Portenta", services=[
    SVC_DEC])
        print("Connecté à", conn.device)
        try:
            mtu = await conn.exchange_mtu(247)  # négociation : on garde le minimum
            print("MTU négocié :", mtu, "=> charge utile max", mtu - 3, "o")
        except Exception as e:
            print("Échange MTU refusé :", e)
        await conn.disconnected(timeout_ms=None)
```

— Dans nRF Connect, la valeur du MTU apparaît dans le menu de l'appareil (*Request MTU*) : le téléphone peut aussi la demander lui-même.

— Vérifiez que les messages JSON arrivent désormais **complets**.

Question

Quel MTU obtenez-vous avec votre téléphone ? Comparez avec vos camarades : Android accepte généralement 247 ou 517, iOS 185 ou 251 selon les modèles. Pourquoi le résultat dépend-il des deux appareils ?

Étape 7 : Mesurer le débit de notifications

On ajoute une commande `BENCH:N` qui envoie N notifications de taille maximale aussi vite que possible, puis on mesure la durée côté Portenta.

```
async def bench(conn, n):
    taille = conn.mtu - 3 if hasattr(conn, "mtu") and conn.mtu else 20
    paquet = bytearray(taille)
    t0 = time.ticks_ms(); erreurs = 0
    for i in range(n):
        paquet[0] = i & 0xFF
        try:
            decision.notify(conn, paquet)      # notifie ce client précis
        except Exception:
            erreurs += 1
        await asyncio.sleep_ms(0)
    dt = time.ticks_diff(time.ticks_ms(), t0)
    debit = n * taille * 8 / dt                # bit/ms = kbit/s
    print("BENCH %d x %d o en %d ms : %.1f kbit/s, %d erreurs" % (n, taille, dt,
    debit, erreurs))
    decision.write(json.dumps({"bench_kbps": round(debit, 1), "err": erreurs}).encode
    (), send_update=True)
```

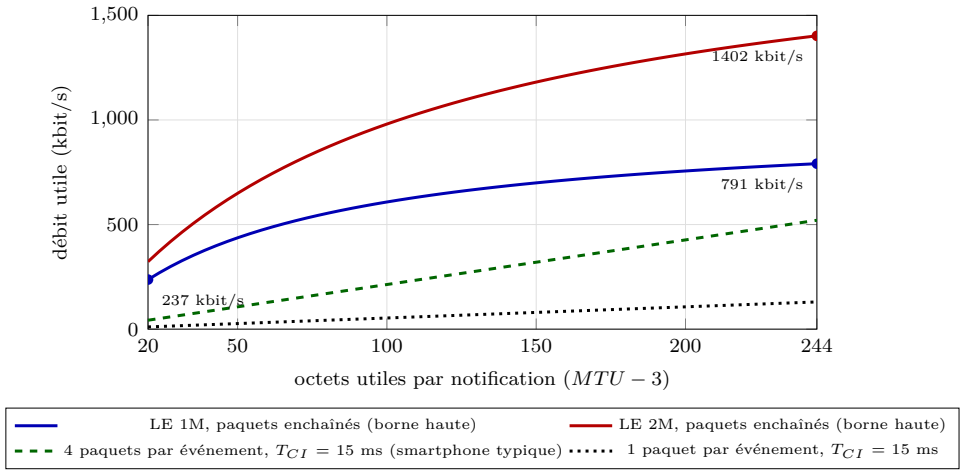
Dans la fonction `commandes()`, ajoutez la branche :

```
        elif cmd.startswith("BENCH:"):
            await bench(conn, int(cmd[6:]))
            continue
```

1. Lancez `BENCH:200` avec le MTU par défaut, puis avec le MTU négocié. Désactivez l'affichage du frame buffer dans l'IDE et suspendez la tâche `vision()` pour une mesure propre.
2. Comptez dans le journal (*Log*) de nRF Connect le nombre de notifications réellement reçues.

Configuration	Charge (o)	Durée (ms)	Débit (kbit/s)	Reçues / envoyées
MTU 23	20			
MTU négocié = …				
MTU négocié, vision active				

Tableau à compléter (il sera repris dans le TP3).



Bornes théoriques vues en cours. Placez vos mesures sur ce graphique : à quelle distance de la borne êtes-vous, et quel est, selon vous, le goulot (radio, téléphone, MicroPython) ?

Étape 8 : Relier les mesures au cours

1. À partir de votre débit mesuré, combien de temps faudrait-il pour transférer une image 96×96 en niveaux de gris (9216 o) ? Et un message de décision ?
2. Avec T_{CI} = 30 ms (valeur courante d'un téléphone), quelle est la latence maximale entre la capture et la réception de la décision sur le téléphone ? Respecte-t-elle l'échéance de 600 ms de la station ?
3. Si l'opérateur s'éloigne et que la connexion tombe, que devient la décision ? Proposez un mécanisme (file locale, renvoi à la reconnexion, publication sur MQTT).

Astuce

Pour un programme autonome, enregistrez le script complet avec **Tools → Save open script to OpenMV Cam (as main.py)**. La Portenta publiera dès la mise sous tension, sans PC.

4 Pour aller plus loin

— Remplacez le JSON par un message binaire de 11 octets (`struct.pack("<BHIBBH", ...)`) et vérifiez qu'il passe sans négocier le MTU.

— Publiez la dernière classe dans les **données fabricant** de la publicité (paramètre `manufacturer` de `aioble.advertise`) : un smartphone la voit sans se connecter (rôle broadcaster).

— Protégez COMMANDE : créez la caractéristique avec une permission exigeant un lien chiffré et observez la demande d'appairage dans nRF Connect (`aioble.config(bond=True)`, selon la version du firmware).

— Documentation OpenMV, module `aioble` : docs.openmv.io ; exemples officiels dans micropython-lib.

— K. Townsend et al., *Getting Started with Bluetooth Low Energy*, O'Reilly, 2014, chapitres 3 (GAP) et 4 (GATT).

— Nordic Semiconductor, guide de nRF Connect for Mobile et cours *Bluetooth Low Energy Fundamentals* (Nordic Developer Academy).